Lutte contre la contrefaçon des billets en euro

Organisation nationale de lutte contre le faux monnayage (ONCFM)

Contexte et objectif

- **Contexte:** Lutte contre le faux-monnayage des billets en euros

- **Objectif**: Prédire grâce au machine learning le modèle avec le taux de prédictions le plus élevé

- **Comment cela fonctionne :**
  - Récupération des caractéristiques du billet après scan
  - Prédiction de la nature du billet (vrai ou faux) basée sur des données déjà existantes

Données à utiliser

- 1500 billets avec leurs caractéristiques et leurs natures:
  - **1000** billets sont **VRAIS**
  - **500** billets sont **FAUX**

- **Déséquilibre des classes** => métrique à adapter

Traitements et analyses

- Assurer la **qualité des données** (nettoyage, lignes dupliquées, valeurs manquantes, etc.)
- Détection d'éventuelles **valeurs aberrantes** soit justifiées soit associées à des erreurs de saisies
- **Prédire les données manquantes** dans le champs "margin_low"
- Sélection du **meilleur modèle**
- Construire toute l'**application de détection des faux billets**

Prédiction des données manquantes sur “margin_low”

- Champs cible de **format numérique**
- Corrélations linéaires avec les autres données
- **Données normalement distribuées** avec quelques outliers assez proches (< 1.5%)
- Séparer le **champs cible du reste des données**
- Meilleur modèle évalué via son score: **Régression linéaire**

Sélection du meilleur modèle

Pré-traitement des données

- Champs cible **“is_genuine”** de type booléen
- **Séparation du champs cible** du reste des données
- Séparation des **données entrainés** et des **données de test**
- Mise à **l’échelle des données** (standardisation normale):
  - moyenne proche de la médiane: outliers peu influents
  - données normalement distribuées
- Intégration de chaque **modèle dans un pipeline**

Définition des modèles à utiliser

- Modèles de type **“classification”** pour une donnée non-numérique

- Modèles de type supervisée (entrainement du modèle en fonction d’un **champs cible**):
  - **Logistic Regression,**
  - **KNN,**
  - **Decision Tree Classifier**,
  - **Random Forest Classifier**

- Modèles de type non-supervisée (clustering des données en fonction de caractéristiques similaires):
  - **K-Means:** regroupement de chaque ligne en fonction de leur proximité avec un centre de gravité

Modèles supervisés
Evaluation entrainement

- Définition des paramètres à évaluer pour chaque modèle
- Evaluation de chaque modèle via l'algorithme **GridSearchCV et la métrique f1 (+ précis car la répartition entre les vrais et faux billets est différente)**
- Récupération de tous les résultats et sélection du meilleur modèle et ses indicateurs en fonction de son score

Modèle supervisé
Résultats entrainement

- **KNeighborsClassifier (KNN)**
- **Taux de prédictibilité:** 99,44%

Modèle supervisé
Evaluation test

- Récupération du meilleur modèle d'entrainement: **KNeighborsClassifier (KNN)**

- Evaluation du score via une **matrice de confusion**
  - Nombre de **faux négatifs et de faux positifs** -> prédiction fausse
  - Nombre de **vrai négatifs et positifs** -> prédiction juste

- Résultats **"entrainement" et "test"** doivent normalement être **assez proche** sinon cela pourrait être problématique

Modèle supervisé
Résultats test

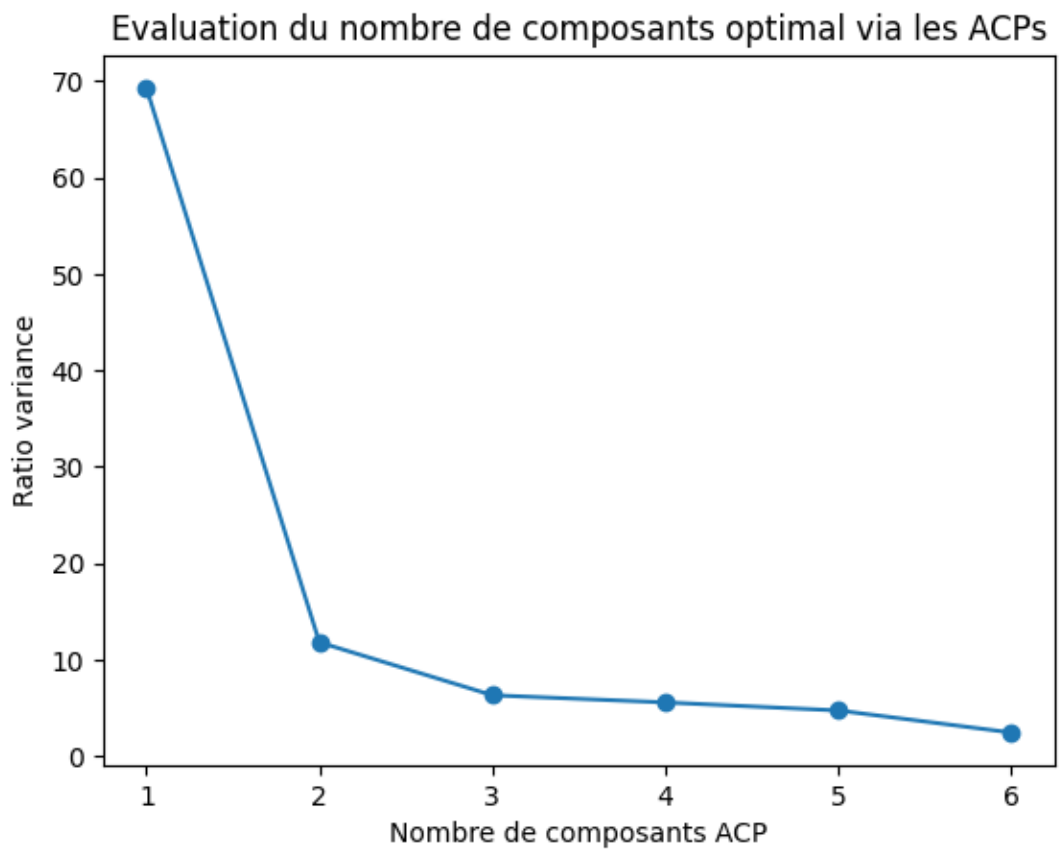
- **Taux de prédictibilité**: 98.75%

- Résultats sur le jeu de données test:
  - **Nombre de faux positifs et négatifs** : 5
  - **Nombre de vrai positifs et négatifs**: 295

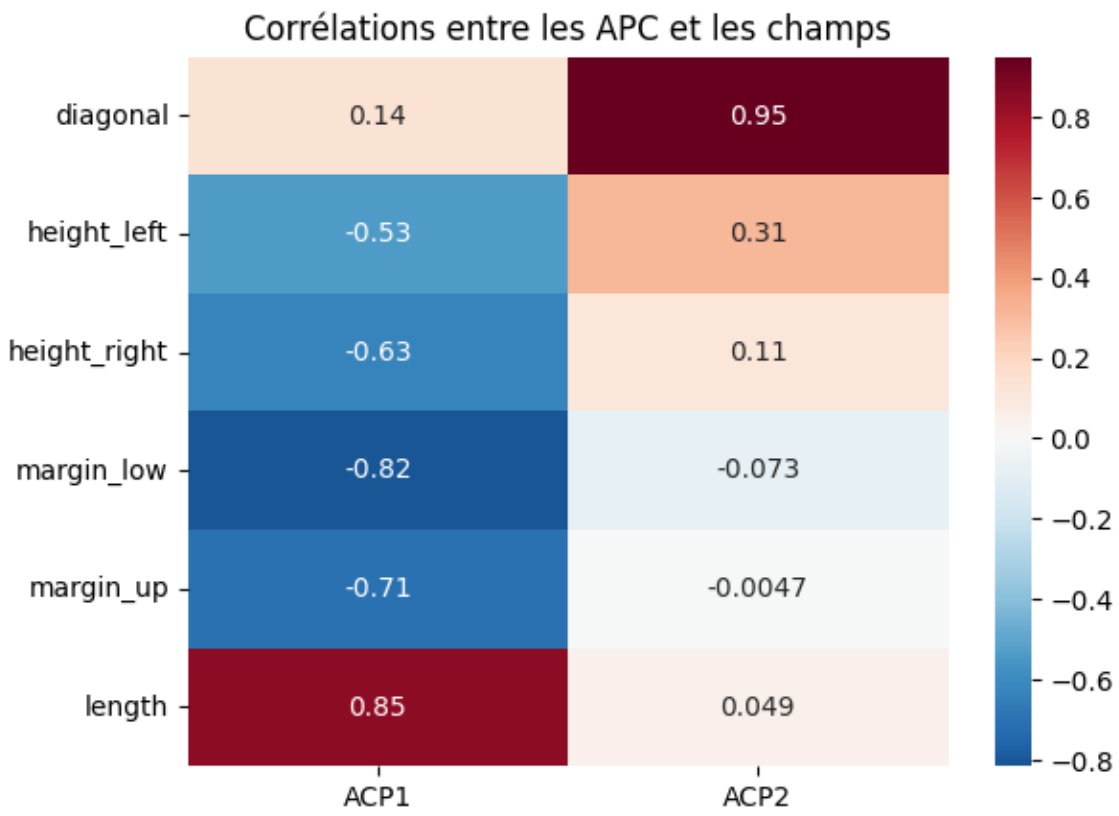
- Résultats entrainement VS test:
  - **entrainement**: 99.44%
  - **test**: 98.75%

Modèle non supervisée (KMeans)

Interprétations des ACPs



2 ACPs > 80% des données



ACP1: Billets longs et peu margés
ACP2: Billets influencés par la diagonale

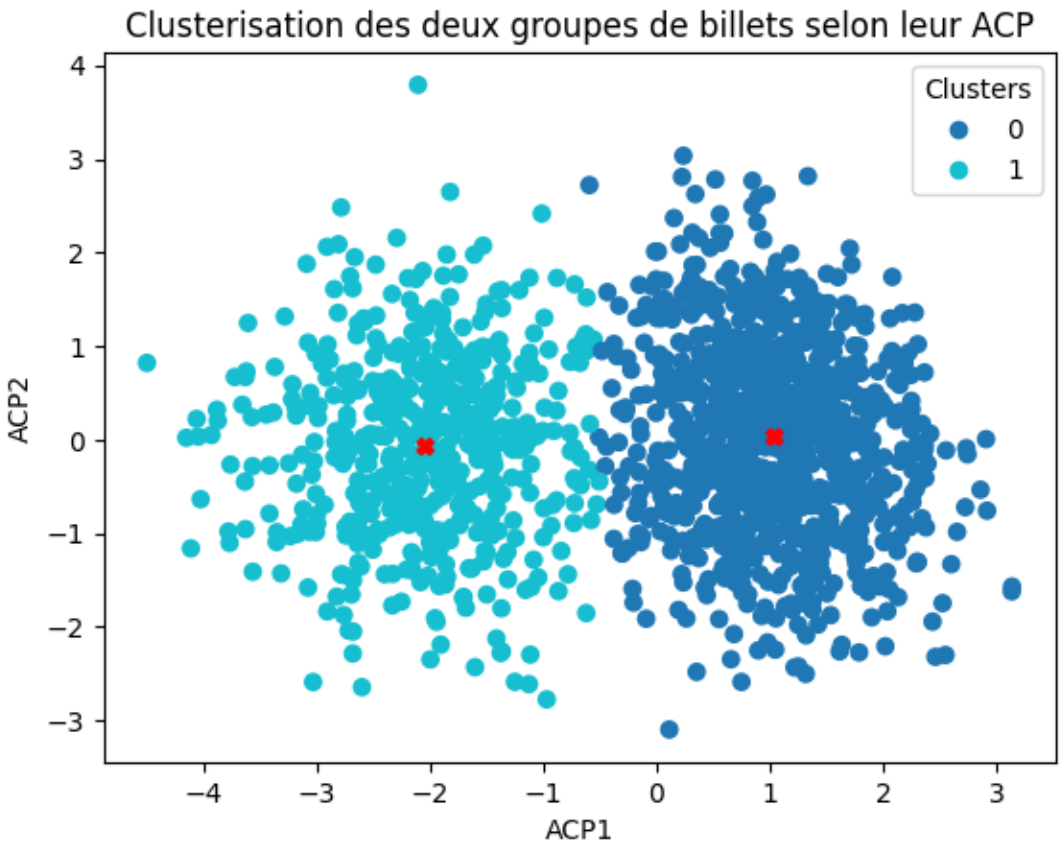
Modèle non supervisée (KMeans)

Résultats

- **2 clusters** (évaluation score silhouette)

- 1er cluster: **997 billets évalués**
  - Billets **longs et peu margés (centroides)**
  - 985 vrais billets / 12 faux billets
  - **98.80% de vrais billets**

- 2nd cluster: **503 billets évalués**
  - Billets **courts avec des marges plus élevées (centroides)**
  - 15 vrais billet / 488 faux billets
  - **2.98% de vrais billets**



KNN vs KMeans

KNN - meilleur choix logique

- Existence d’un champs cible
- Meilleur apprentissage des **relations entre features** et **champs cible**
- Prédictions plus **fiables** et basés sur un **large panel de métriques**
- **Généralisation** du modèle sur de **nouveaux jeux de données**
- Modèle non supervisé (KMeans) utile pour détecter **comment les données se regroupent selon des caractéristiques communes** et **explorer la structure des données**

Application de détection des faux billets

Construction du script

- Sauvegarde du **meilleur modèle (format “.pkl”)** issu du notebook
- Création d’un fichier **“main.py”**
- Prédictions **basées sur le modèle retenu** et **le fichier CSV donné**
- Génération d’un **nouveau fichier CSV avec les prédictions**

Exemple de fichier CSV à tester

id	diagonal	height_left	height_right	margin_low	margin_up	length
A_1	171.76	104.01	103.54	5.21	3.3	111.42
A_2	171.87	104.17	104.13	6.0	3.31	112.09
A_3	172.0	104.58	104.29	4.99	3.39	111.57
A_4	172.49	104.55	104.34	4.44	3.03	113.2
A_5	171.65	103.63	103.56	3.77	3.16	113.33

Aperçu du fichier M.A.J csv généré

id	diagonal	height_left	height_right	margin_low	margin_up	length	predictions
A_1	171.76	104.01	103.54	5.21	3.3	111.42	False
A_2	171.87	104.17	104.13	6.0	3.31	112.09	False
A_3	172.0	104.58	104.29	4.99	3.39	111.57	False
A_4	172.49	104.55	104.34	4.44	3.03	113.2	True
A_5	171.65	103.63	103.56	3.77	3.16	113.33	True